Real-Time Zebra Crossing Detection on a Sub-Watt FPGA Camera System: Hardware Results and Comparison with GPU, CPU and Mobile Approaches

Raj Mali
Dept. of Electronics and Computer Science
Vidyalankar Institute of Technology
Mumbai, India
raj.mali@vit.edu.in

Pooja Nagvekar
Dept. of Electronics and Telecommunication
Vidyalankar Institute of Technology
Mumbai, India
pooja.nagvekar@vit.edu.in

Prof. Anuradha Joshi
Dept. of Electronics and Computer Science
Vidyalankar Institute of Technology
Mumbai, India
anuradha.joshi@vit.edu.in

Prof. Amaya Pethe
Dept. of Electronics and Computer Science
Vidyalankar Institute of Technology
Mumbai, India
amaya.pethe@vit.edu.in

***Abstract*—We present and evaluate a complete camera-to-decision zebra crossing detector on a low-cost Xilinx Artix-7 XC7A35T FPGA. An OV7670 camera feeds an in-fabric pipeline—grayscale conversion, 8×8 block averaging to 80×60 pixels, a histogram-based auto-contrast stage and a four-layer binarized convolutional neural network (BCNN) with 93,408 one-bit weights—whose decision drives a VGA overlay, a character LCD, LEDs and a buzzer with no processor in the loop. On the board the network evaluates every camera frame (27 decisions/s, 34.0 ms per inference at 66.7 MHz); the complete system uses 3,370 LUTs (16 %), 19 block-RAM tiles and one DSP slice at an estimated 0.29 W of on-chip power, of which the network core takes 22 mW. Sending the 639 images of two held-out test sets through the physical FPGA reproduced the bit-exact software model on 639 of 639 images and the published accuracy of 79.4 % on dashcam frames and 79.1 % on street photographs from unseen photographers, with 95.7 % of the confident answers correct. We show that the real sensor's flat images reduce accuracy to 47.7 % and 67.0 % on these sets and that the hardware auto-contrast stage restores 78.2 % and 78.3 %. Compared with a YOLOv5 detector on a Jetson Nano, and with the same BCNN and a ResNet-18 on a laptop CPU, the FPGA system needs an estimated 11 mJ per decision instead of 302 mJ to 1.9 J while running at the camera frame rate.**

***Index Terms*—binarized neural networks, FPGA, zebra crossing detection, embedded vision, energy efficiency, hardware-in-the-loop verification**

I. Introduction

Zebra crossing detection supports navigation aids for visually impaired pedestrians and driver-assistance functions. Both applications need a detector that is small, cheap, low-power and fast enough to keep up with a camera. Published systems meet the speed requirement on smartphones [7], [8] or on embedded GPU modules [9], which draw several watts and need an operating system. Binarized neural networks (BNNs) [1], [2] replace multiply-accumulate with XNOR and popcount and map naturally to FPGA logic [3], but FPGA BNN accelerators are usually built for throughput on large devices and receive images from a host processor rather than from a camera.

In earlier work [11] we designed a BCNN accelerator for this task and verified it in simulation only. This paper reports what happened when that accelerator became a complete camera system on the smallest Artix-7 device, measured on the board, and compares the result with mobile, GPU, CPU and FPGA alternatives. The contributions are results:

1) A processor-free camera-to-decision system that classifies **every** frame of a 26.7 fps OV7670 camera on a low-cost XC7A35T FPGA, using 16 % of its LUTs and an estimated 0.29 W of on-chip power.
2) Hardware-in-the-loop evidence: all 639 held-out test images are classified bit-exactly by the physical FPGA, reproducing the software accuracy on hardware.
3) A quantified camera-domain study: the sensor's flat images cost up to 31 accuracy points; a 0.15 ms hardware auto-contrast stage removes the loss.
4) A comparison with smartphone, Jetson Nano, laptop-CPU and FPGA-BNN approaches on rate, power, energy per decision and area, including where this design is not better.

II. Related Work and Positioning

Zebra crossing detection. Early methods used the periodic stripe pattern and vanishing-point geometry [6]. ZebraRecognizer [7] combines image processing with the phone's inertial sensors and reports precision 1.0 and recall 0.93 at about 25 fps on an iPhone 5S. LYTNet [8], a MobileNetV2-derived CNN, classifies pedestrian traffic lights with 94 % accuracy and estimates the crossing direction with a 6.35° mean angle error at 20 fps on an iPhone 7. CDNet [9] adapts YOLOv5 with channel attention, negative-sample training and region-of-interest cropping and reports an F1 score of 94.72 % at 33.1 fps on a Jetson Nano with TensorRT. All three need a phone or a GPU module.

Binarized networks on FPGAs. FINN [3] streams a CIFAR-10 BNN at 21.9 k images/s and 80.1 % accuracy on a Zynq-7045, using 82,988 LUTs and 396 block RAMs at 8.8 W chip power, with images supplied by the on-chip ARM host. Our design takes the opposite point of the design space: one shared engine processes one kernel tap per clock, which fits the smallest Artix-7 with room for the camera, display and interface logic, and needs no host.

III. System Overview

Fig. 1 shows the system. The OV7670 [14] delivers 640×480 RGB565 frames at 26.7 fps from a 21.33 MHz clock. A preprocessing block converts each pixel to gray as $(77R + 150G + 29B) \gg 8$, exactly as in training, sums 8×8 blocks and shifts by six to obtain an 80×60 image, which is written into one of two alternating frame-buffer banks. A frame is accepted only if all 480 lines carried at least 640 pixels. Block sums are accumulated as pixels arrive, so no full-resolution frame is stored for detection.

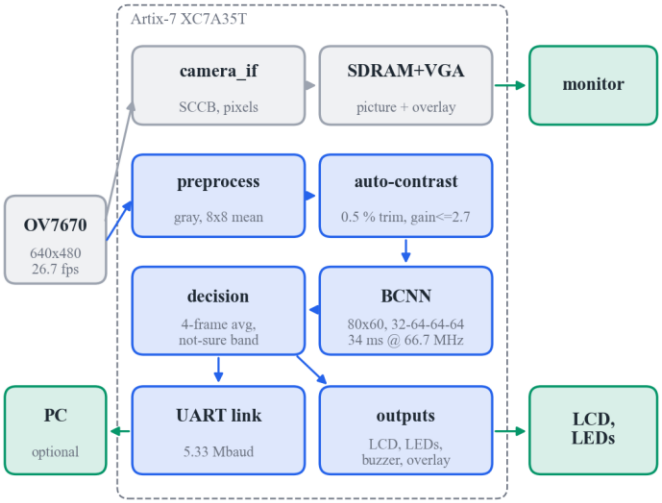


Fig. 1. Architecture. Grey: hardware-proven OV7670–SDRAM–VGA design (kept); blue: new detection path, which reads the camera pins directly and does not depend on the SDRAM; green: outputs.

Auto-contrast. For every frame a 256-bin histogram gives lo and hi, the 25th darkest and brightest pixels (a 0.5 % trim, so a lamp or a dark corner does not defeat the stretch). Each pixel becomes $p' = \min(255, (\max(p - lo, 0) \cdot \lfloor 65280 / \max(hi - lo, 96) \rfloor) \gg 8)$. The divisor floor of 96 caps the gain at 2.7 so that flat surfaces are not blown up into stripe-like texture. Histogram, levels, a 16-step division and the stretched copy into the network together take about 10 k clock cycles (0.15 ms).

BCNN. The network has four binary 3×3 convolution layers with 32-64-64-64 channels, each followed by an integer threshold (folded BatchNorm) and 2×2 max-pooling, and a 960-input binary output neuron [11]. A single pipelined engine evaluates one kernel tap per clock, 2,269,477 cycles per image, i.e. 34.0 ms at 66.7 MHz, which is shorter than the 37.5 ms camera frame period.

Decision and outputs. The margin (score minus the threshold of 477) is averaged over four frames and classified as crossing, not sure or no crossing with the band $-31 \le \text{margin} < 33$, chosen on validation photographs only [11]. The decision drives a VGA overlay (Fig. 2), a 16×2 LCD, LEDs with an eight-LED confidence meter and a buzzer. A 5.33 Mbaud UART link provides snapshots, photo upload and status.

Clocks and verification. One MMCM generates 133.33 MHz (camera, SDRAM, UART), 66.7 MHz (BCNN), 25 MHz (VGA) and the 21.33 MHz camera clock. Every crossing between clock domains uses a dual-clock block RAM or a toggle handshake. Each new block was verified bit-exactly against a Python golden model in simulation, including a real frame captured from the board's OV7670, and the whole design was simulated with Xilinx primitive models before it was programmed.

IV. Experimental Setup

Hardware. EDGE Artix-7 board (XC7A35T-1FTG256C), OV7670 camera module, VGA monitor and 16×2 LCD; Vivado 2024.2. Power is the Vivado post-route estimate of the whole chip (vectorless), not a supply measurement.

Model and data. The deployed model is the v3 BCNN of [11], trained on 11,281 images from 14 sources with knowledge distillation [4] from a full-precision ResNet-18 [5]. T1 contains 524 dashcam frames from CDSet [10]; T2 contains 115 hand-checked street photographs by photographers excluded from training.

Hardware-in-the-loop. Each test image is sent over the UART into a dedicated frame-buffer bank, classified by the FPGA, and the returned score is compared with the integer golden model.

CPU baseline. Intel Core i5-12500H (12 cores, 45 W processor base power [13]), PyTorch, batch size 1, median of 200 runs. The ResNet-18 is the full-precision ceiling model of [11] on the same input.

Camera conditions. Test images were altered at 80×60 resolution: contrast scaled by 0.6, a camera-like contrast of 0.4 around a mean of 115 (matching the span of a real frame, Fig. 5), the same with a saturated 3×3 bright and dark block ("lamp"), gamma 0.6 and 1.6, a 0.7-pixel Gaussian blur and sensor noise with $\sigma = 4$.

V. Results

A. Operation on the Board

TABLE I
Measured on the EDGE Artix-7 board

Check	Result
Self-test (8 checks)	all pass
Camera frames committed	27.5 fps, 640×480, 0 rejected
BCNN on live camera	27 inferences/s, 0 skipped
T2 street photos through the FPGA	115/115 bit-exact, 79.13 %
T1 dashcam frames through the FPGA	524/524 bit-exact, 79.39 %
Photo round trip over UART	48.2 ms per image
Live: road without a crossing	NO CROSSING, margin −73
Live: crossing photo on a screen	CROSSING, margin up to +49
VGA overlay, LCD, LEDs	as designed

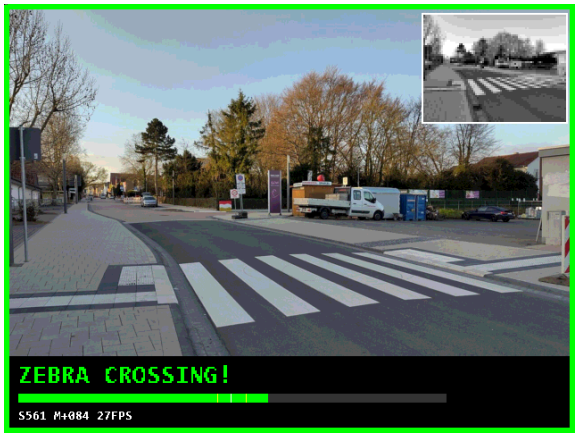


Fig. 2. VGA output for a detected crossing: border colour, the 80×60 network input (top right), decision, score bar with threshold and not-sure band, and score, margin and rate. Simulated RTL of the VGA path with a street photograph as the camera input.

Table I summarises the board results. The first programming of the bitstream passed every self-test check. The 639 test images sent through the physical FPGA produced exactly the scores of the bit-exact integer model, so the hardware reproduces the accuracy measured in software on both test sets. The camera path evaluated every frame; pointed at a road without a crossing the system answered no crossing with a margin of −73, and pointed at a crossing photograph shown on a screen it

reached margins of up to +49 and displayed the crossing state on the monitor, LCD and LEDs.

B. Accuracy and Confident Answers

TABLE II
ACCURACY ON THE HELD-OUT TEST SETS

Model	Weights	T1	T2
BCNN v1, 40×30, one source [11]	14 kbit	82.1 %	61.7 %
BCNN v3, 80×60 (deployed)	93 kbit	79.4 %	79.1 %
v3, confident answers†		90.2 %	95.7 %
ResNet-18, full precision	11.2 M	81.1 %	95.7 %

Table II places the deployed BCNN between a single-source baseline and a full-precision ResNet-18 on the identical input. On street photographs from unseen photographers the binary network reaches 79.1 %, 17.4 points above the single-source model and 16.6 points below the full-precision ceiling, so binarization rather than input resolution limits accuracy († not-sure band applied; 51 % of T1 and 40 % of T2 answered).

TABLE III
DECISIONS OF THE FPGA ON THE HELD-OUT SETS

	T1 dashcam	T2 street
Images (with a crossing)	524 (292)	115 (67)
True / false detections	197 / 13	56 / 13
Missed / true rejections	95 / 219	11 / 35
Precision / recall	93.8 / 67.5 %	81.2 / 83.6 %
F1 / accuracy	78.5 / 79.4 %	82.4 / 79.1 %
Answered outside the band	266 (50.8 %)	46 (40.0 %)
Correct among answered	240 (90.2 %)	44 (95.7 %)
Errors removed by the band	82 of 108	22 of 24

Table III lists the decisions returned by the FPGA, which are identical to those of the software model. On the street photographs the classifier makes 13 false and 11 missed detections (precision 81 %, recall 84 %, F1 82 %); on the dashcam frames it is conservative (precision 94 %, recall 67 %). Fig. 3 shows why the "not sure" band works: most of the overlap between the margins of the two classes falls inside the band from −31 to +33, so withholding an answer there removes 82 of the 108 errors on T1 and 22 of the 24 errors on T2. Of the 46 street photographs that receive an answer, 44 are correct. Fig. 4 generalises this to any coverage: ranking the answers by their distance from the band centre trades the share of images answered against the accuracy of those answers.

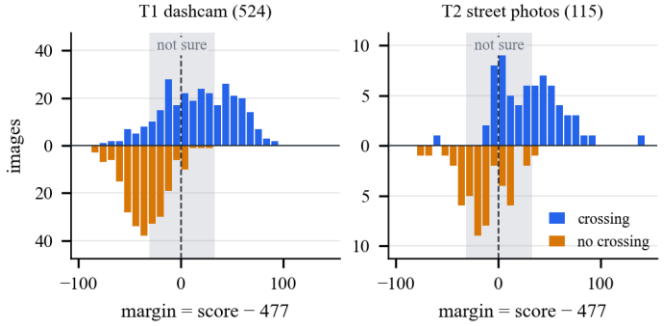


Fig. 3. Margin (score − 477) of every held-out image as returned by the FPGA; crossings above and non-crossings below the axis. Dashed: decision threshold; grey: not-sure band.

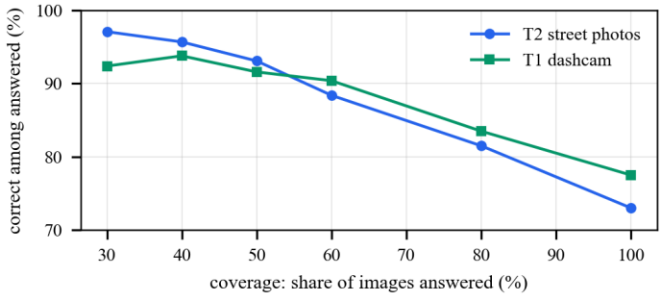


Fig. 4. Selective prediction on the camera path (with auto-contrast): accuracy of the answers versus the share of images answered, ranked by the distance of the margin from the band centre.

C. The Real Camera and Auto-Contrast



Fig. 5. (a) A frame from the board's OV7670, (b) its 80×60 network input spans only gray levels 58–148, (c) after hardware auto-contrast.

A frame captured from the board's camera spans only gray levels 58 to 148 after reduction to 80×60 (Fig. 5): the sensor setting produces flat, washed-out pictures that the network never saw in training. Fig. 6 and Table IV quantify the effect. With camera-like contrast the accuracy collapses to 47.7 % (T1) and 67.0 % (T2); the hardware auto-contrast restores 78.2 % and 78.3 %, and the trimmed histogram keeps the gain when a lamp is in view, where a plain min–max stretch fails. The gain cap matters on flat surfaces: a real frame of a ceiling scored a margin of +29 (stripe-like) with an unlimited gain and −11 with the cap. The price is a small loss on already well-exposed photographs (79.4/79.1 % to 77.5/73.0 %), which is why photographs uploaded from a PC bypass the stage.

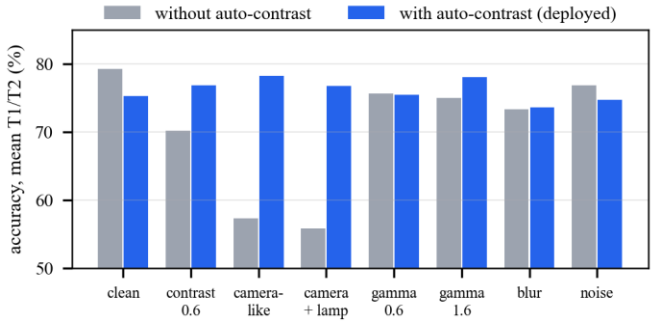


Fig. 6. Accuracy (mean of T1 and T2) under camera effects, without and with the hardware auto-contrast, bit-exact integer model.

TABLE IV
AUTO-CONTRAST VARIANTS (T1 / T2 ACCURACY, %)

Stretch	Clean	Camera-like	With lamp
None	79.4 / 79.1	47.7 / 67.0	47.3 / 64.3
Min–max	77.9 / 77.4	78.2 / 78.3	47.3 / 64.3
0.5 % trim, no cap	77.5 / 73.0	78.1 / 77.4	77.5 / 75.7
0.5 % trim, gain ≤ 2.7	77.5 / 73.0	78.2 / 78.3	77.5 / 75.7

D. FPGA Implementation

TABLE V
RESOURCES ON THE XC7A35T (ROUTED)

Block	LUT	FF	BRAM	DSP
BCNN core	744	321	10	0
Preprocessing	211	257	0	0
Auto-contrast, loader, decision	615	593	0	1
Frame buffers, UART link	244	132	7	0
Camera, SDRAM, VGA, capture, UART	1,069	769	1.5	0
Overlay, LCD, text, buzzer, clocks	490	658	0.5	0
Complete system¶	**3,370**	**2,730**	**19**	**1**
Share of the device	16.2 %	6.6 %	38 %	1.1 %

Table V breaks down the complete design (block RAM in 36-kbit tiles; ¶ the rows come from the hierarchical report, where LUT sharing makes their sum three LUTs larger than the total). The BCNN core occupies 744 LUTs and no DSP slices; the whole detection path including auto-contrast and the UART link needs 1,814 LUTs. All clocks meet timing after routing, with worst setup slack of +0.835 ns at 133.33 MHz, +2.616 ns at 66.7 MHz and +28.1 ns at 25 MHz. The SDRAM interface, whose I/O registers were moved into the I/O blocks, meets setup and hold checks derived from SDRAM data-sheet numbers; its output delays fell from 4.0–8.2 ns in the original camera design to 1.6–4.2 ns.

E. Power, Speed and Energy

TABLE VI
ESTIMATED ON-CHIP POWER BY BLOCK

Block	Power	Share
BCNN core	22 mW	7.5 %
Preprocessing	3 mW	1.0 %
Auto-contrast, loader, decision	8 mW	2.7 %
Frame buffers, UART link	13 mW	4.4 %
Camera, SDRAM, VGA, capture	13 mW	4.4 %
Overlay, LCD, text, buzzer	3 mW	1.0 %
Clock generation (MMCM)	109 mW	37.1 %
I/O buffers, top-level logic	50 mW	17.0 %
Device static	73 mW	24.8 %
Complete chip	**294 mW**	**100 %**

Table VI splits the Vivado estimate of 294 mW (routed design, vectorless activity, so indicative only) by block. The network itself is a small part of it: the BCNN core accounts for 22 mW, i.e. 0.75 mJ per 34.0 ms inference, and the whole detection path including preprocessing, auto-contrast, frame buffers and the UART link for 46 mW, or 1.7 mJ per camera frame. Clock generation (109 mW for one MMCM with an 800 MHz VCO) and device static power (73 mW) dominate, so the system energy per decision is set by the platform rather than by the network, and the clocking is the first place to look for further savings.

TABLE VII
SPEED AND ENERGY PER DECISION

Implementation	Time	Rate	Power	Energy
This work, FPGA	34.0 ms	27 /s	0.29 W†	**11 mJ**
CDNet, Jetson Nano [9]	—	33.1 /s	10 W‡	302 mJ
Same BCNN, i5-12500H	8.1 ms	123 /s	45 W‡	366 mJ
ResNet-18, i5-12500H	42.5 ms	24 /s	45 W‡	1.9 J

Table VII and Fig. 7 compare energy per decision. † Vivado estimate for the complete chip including camera, SDRAM and display logic; ‡ rated module or processor base power [12], [13], used as an estimate because neither platform was metered. A laptop CPU runs the same BCNN about four times faster per image, but at 45 W instead of 0.29 W; per decision the FPGA system needs roughly 30 times less energy than the CPU or the Jetson Nano and about 175 times less than the full-precision ResNet-18, while still keeping up with the camera.

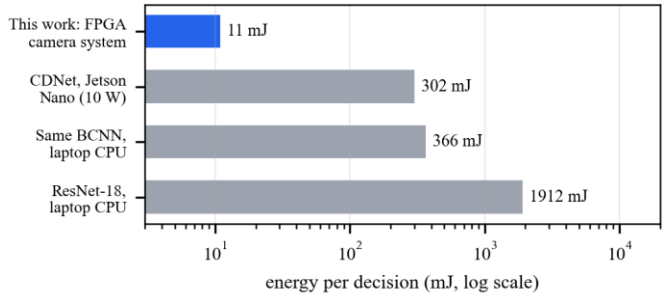


Fig. 7. Estimated energy per decision (log scale), from the power and rates in Table VII.

F. Model Variants for the Camera

We also retrained the v3 recipe with camera augmentation (gamma, focus blur and sensor noise; v3c) and on auto-contrasted inputs (v3cs). Through the camera path v3c is about three points more accurate as a plain yes/no answer (80.7 % on T1 and 76.5 % on T2 against 77.5 % and 73.0 %), and both variants match v3 on the average of nine camera effects (77.1 % against 76.9 %). When only the confident answers count, however, v3 is better on street photographs (95.7 % against 89.1 % at 40 % coverage), so v3 remains the deployed model; with 115 photographs the differences amount to a few images.

VI. COMPARISON WITH EXISTING SYSTEMS

TABLE VIII
COMPARISON WITH PUBLISHED SYSTEMS

System	Platform	Rate	Reported accuracy§	Power
ZebraRecognizer [7]	iPhone 5S	≈25 fps	P 1.0, R 0.93	phone
LYTNet [8]	iPhone 7	20 fps	94 % (lights)	phone
CDNet [9]	Jetson Nano	33.1 fps	F1 94.72 %	5–10 W
FINN CNV [3]	Zynq-7045 + ARM host	21.9 k/s	80.1 % CIFAR-10	8.8 W chip
This work	**XC7A35T only**	**27 fps**	**79.1 % T2**	**0.29 W**

Table VIII compares the system with published work. § Each accuracy refers to the authors' own data and task, so the numbers cannot be ranked directly; the comparison concerns what each platform needs to deliver its result.

Where this design is better. (i) Energy and power: at an estimated 0.29 W the complete camera system draws more than an order of magnitude less than the 5–10 W of the Jetson-based detector, and needs no operating system, phone or GPU module. (ii) Area: the BCNN core uses 744 LUTs, about 110 times fewer than the FINN CNV prototype, so the smallest Artix-7 holds the camera interface, frame store, display and interfaces with 84 % of its LUTs still free. (iii) Integration: the camera is connected directly to the fabric; prior FPGA BNN accelerators are fed by a host processor. (iv) Verifiability: the published test sets were run on the physical device with bit-exact agreement,

and the not-sure band makes confident answers 95.7 % correct on street photographs. (v) Robustness to the real sensor is measured and corrected in hardware.

Where it is not. CDNet and LYTNet report higher accuracy on their own data, and a full-precision ResNet-18 reaches 95.7 % on our street photographs; the binary network is the accuracy bottleneck. FINN processes small CIFAR images about 800 times faster; our engine trades throughput for area and only needs to match the camera frame rate. The CPU runs the BCNN faster per image, at roughly 150 times the power.

VII. Discussion and Limitations

Measure the sensor first. The largest single effect on accuracy came not from the network but from the camera: one captured frame revealed a contrast span that cost up to 31 points, and a small hardware stage removed most of the loss. A domain check with real sensor data belongs at the start of an edge-vision design.

Hardware-in-the-loop on the published test set turned simulation-only evidence into a measured result at almost no cost: 639 images took under a minute through the UART link.

Limitations. Power is a tool estimate and the CPU and GPU energies use rated power, not measurements. Accuracies across published systems refer to different datasets. T2 has 115 images, so its 95 % confidence interval is about ±7 points. The live camera tests are qualitative; a systematic on-scene evaluation and fine-tuning on frames from the target camera, for which the tools exist, are future work.

VIII. Conclusion

We built and measured a processor-free zebra crossing detector on a low-cost Artix-7 FPGA that takes frames from an OV7670 camera, evaluates a binarized CNN on every frame and shows its decision on a monitor, an LCD and LEDs. On the board it reproduced the bit-exact software results on all 639 test images and the published accuracy of 79.4 % and 79.1 %, with 95.7 % of confident answers correct on street photographs. A hardware auto-contrast stage recovered up to 31 accuracy points lost to the real sensor. The complete system uses 16 % of the device's LUTs and an estimated 0.29 W, about 11 mJ per decision, 27 to 175 times less than the GPU and CPU alternatives we compared. Future work will fine-tune on camera data from the target scene and use the free logic for channel-parallel inference.

References

[1] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training neural networks with weights and activations constrained to +1 or −1," arXiv:1602.02830, 2016.

[2] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: ImageNet classification using binary convolutional neural networks," in Proc. ECCV, 2016.

[3] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "FINN: A framework for fast, scalable binarized neural network inference," in Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays, 2017.

[4] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," arXiv:1503.02531, 2015.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016.

[6] S. Se, "Zebra-crossing detection for the partially sighted," in Proc. IEEE CVPR, 2000.

[7] S. Mascetti, D. Ahmetovic, A. Gerino, and C. Bernareggi, "ZebraRecognizer: Pedestrian crossing recognition for people with visual impairment or blindness," Pattern Recognition, vol. 60, pp. 405–419, 2016.

[8] S. Yu, H. Lee, and J. Kim, "LYTNet: A convolutional neural network for real-time pedestrian traffic lights and zebra crossing recognition for the visually impaired," arXiv:1907.09706, 2019.

[9] Z. Zhang et al., "CDNet: A real-time and robust crosswalk detection network on Jetson Nano based on YOLOv5," Neural Computing and Applications, 2022, doi:10.1007/s00521-022-07007-9.

[10] "CDSet: Crosswalk dataset for zebra crossing detection," Zenodo, record 8289874.

[11] R. Mali, "A bit-exact binarized CNN for zebra crossing detection on a low-cost Artix-7 FPGA," project report, Vidyalankar Institute of Technology, Mumbai, 2026.

[12] NVIDIA, "Jetson Nano Developer Kit," product specification (5 W and 10 W power modes).

[13] Intel, "Intel Core i5-12500H processor," product specification (45 W processor base power).

[14] OmniVision, "OV7670/OV7171 CMOS VGA (640×480) CameraChip sensor," datasheet.